Open Source Formal Verification

engines

Yann Thoma

Reconfigurable and Embedded Digital Systems Institute
Haute Ecole d'Ingénierie et de Gestion du Canton de Vaud



This work is licensed under a Creative Commons Attribution-NonCommercial-ShareAlike 3.0 Unported License

November 2026

1. Introduction
2. Bounded Model Checking
3. Provers
4. Engines in SBY
5. Conclusion

Introduction

- SBY offers various engines
  - Open Source
- Bounded Model Checking and bounded proof
  - Up to a certain depth
- Unbounded proof
  - No depth, but risk of unconclusive proof

Model Checking

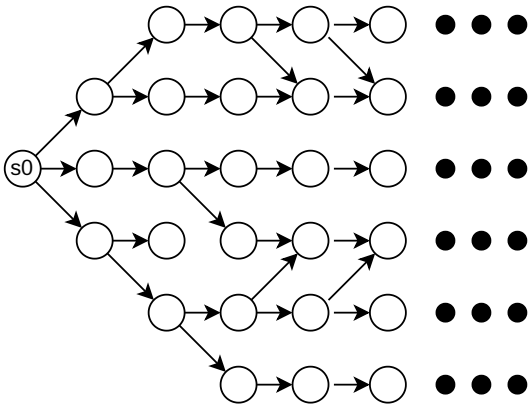
Model checking

Method for checking whether a finite-state model of a system meets a given specification

- Applied to state machines
  - A hardware model is by itself a state machine
    - State represented by the D flip-flops and memories
    - Inputs are simply the inputs of the module
  - The specification is given by properties and invariants

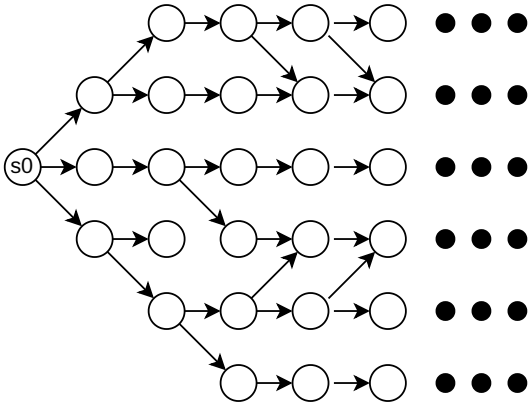
Model Checking

- Concept:
  - From a reset condition
  - Try to find a state where an assertion fails
- Process:
  1. Start from reset and check the assertions
  2. Generate successor states and repeat
- *Counterexample*: a reachable assertion violation
- *Proof*: exhaustive check completes; all reachable states pass
- *Unknown*: cannot conclude (e.g. resource limit)



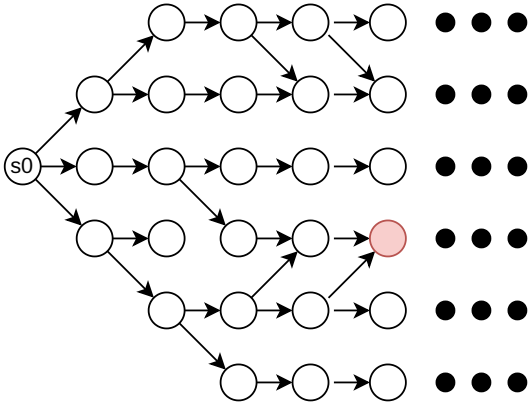
Bounded Model Checking

- Concept:
  - From a reset condition
  - Try to find a state where an assertion fails
  - For at most *N* steps (the depth)
- Process:
  1. Start from reset and check the assertions
  2. Generate successor states and repeat, up to depth *N*
- *Fail*: a violation is found within depth *N*
- *Pass*: no violation through depth *N*; bounded only, not an unbounded proof
- *Unknown*: no conclusion, e.g. timeout or resource limit



Bounded Model Checking

- Red state corresponds to a failing assertion
- Found after 5 cycles
- If the depth is at least 5, then the failure will be detected
- If the depth is less that 5, then the checker will not detect it



What depth?

- Choose depth from the behavior you want the check to reach
  - Count the cycles from reset/start through pipeline latency, protocol handshakes, and the target state
  - Counter example: to test terminal count *M* from zero, allow at least *M* increment cycles plus the cycle that checks the terminal behavior
  - FIFO example: to reach full and then drain, allow at least the fill and drain transactions, plus any protocol latency
  - Include stalls only with a stated bound on how long the environment may stall; an unconstrained stall can make a finite latency bound impossible
- Start with that bound, inspect traces/results, and increase it if the target behavior is not reached
- Greater depth can cost more time and memory; a bounded PASS still says nothing about behavior beyond the bound
- *Good argument*: “as small as possible, but not smaller” (or in French, *“aussi petite que possible, mais aussi grande que nécessaire”*)
- *Good argument for generic parameters*

Bounded search and unbounded proof

- **BMC:** checks executions only up to the configured depth
  - A counterexample within the bound demonstrates a violation
  - No counterexample within the bound is only a bounded PASS, not a proof for all depths
- **Proof engines:** use methods such as induction or PDR to try to establish a property for all reachable executions
  - A completed proof establishes the encoded property for the modeled design under its assumptions
  - The engine may instead find a counterexample, time out, or return UNKNOWN; convergence and termination are not guaranteed
- Choose BMC for bounded bug finding and a proof-capable engine when an unbounded result is needed; both are useful, and resource limits can affect either run

Proof

- BMC is not ideal because of the depth to be chosen
- Can we prove the correctness?
- Option: k-induction

1-induction

For a property P:

$$P(0) \wedge \forall n(P(n) \Rightarrow P(n+1)) \Rightarrow \forall n P(n)$$

k-induction

Base: P holds on every initialized path through the first k states:

$$\bigwedge_{i=0}^{k-1} P(i)$$

Inductive step: any k legal transitions with P in each preceding state imply P in the next:

$$\forall n \left(\left(\bigwedge_{i=0}^{k-1} P(n+i) \right) \Rightarrow P(n+k) \right)$$

If

$$A_k := \left(\bigwedge_{i=0}^{k-1} P(i) \right) \wedge \forall n \left(\left(\bigwedge_{i=0}^{k-1} P(n+i) \right) \Rightarrow P(n+k) \right)$$

K-induction states that:

$$A_k \Rightarrow \forall n P(n)$$

Why use $k > 1$?

- In 1-induction, assuming $P(s_0)$ and one legal transition must prove $P(s_1)$ for every possible s_0
- That step may fail on an unreachable state that does not represent enough history
- k-induction adds a window of k consecutive states and transitions; this can supply the history needed for a property involving delayed effects or state propagation
- A larger k can make the inductive step strong enough, but increases the base-check work and is not guaranteed to converge

Verification

- Proofs and bmc exploit a solver
- The solvers are usually based on SAT or SMT solvers
- SAT (Boolean satisfiability problem)
  - Is there a combination of variables that satisfy a boolean equation in its conjunctive form?
  - $(x1 \vee x3) \wedge (\overline{x2} \vee x3) \wedge (x1 \vee \overline{x3})$
  - NP-complete problem
- SMT (satisfiability modulo theories)
  - Generalizes SAT to more complex formulas involving:
    - Real numbers, integers
    - Lists, arrays, bit vectors
    - ...

Engines in SBY

Mode	Engine
bmc	smtbmc [all solvers] btor btormc btor pono btor rIC3 abc bmc3 aiger aigbmc aiger rIC3
prove	smtbmc [all solvers] abc pdr btor rIC3 aiger avy aiger rIC3 aiger suprove
cover	smtbmc [all solvers] btor btormc

The following a python script allows to test the engines and solvers available in the OSS CAD Suite:



smtbmc solvers (1)

- The SMTBMC engine supports the following solvers:
  - yices (`https://yices.csl.sri.com/`)
    - Yices 2 is an SMT solver that decides the satisfiability of formulas containing uninterpreted function symbols with equality, real and integer arithmetic, bitvectors, scalar types, and tuples.
  - boolector (`https://boolector.github.io/`)
    - A Satisfiability Modulo Theories (SMT) solver for the theories of fixed-size bit-vectors, arrays and uninterpreted functions.
  - bitwuzla (`https://bitwuzla.github.io/`)
    - Bitwuzla is a Satisfiability Modulo Theories (SMT) solver for the theories of fixed-size bit-vectors, floating-point arithmetic, arrays and uninterpreted functions and their combinations.
  - z3 (`https://github.com/Z3Prover/z3`)
    - Z3 is a theorem prover from Microsoft Research.

smtbmc solvers (2)

- The SMTBMC engine supports the following solvers:
  - mathsat (`https://mathsat.fbk.eu/`)
    - MathSAT 5 is an efficient Satisfiability modulo theories (SMT) solver supporting a wide range of theories (including e.g. equality and uninterpreted functions, linear arithmetic, bit-vectors, and arrays) and functionalities.
  - cvc4 (`https://cvc4.github.io/`)
    - CVC4 is an efficient open-source automatic theorem prover for satisfiability modulo theories (SMT) problems. It can be used to prove the validity (or, dually, the satisfiability) of first-order formulas in a large number of built-in logical theories and their combination.
  - cvc5 (`https://cvc5.github.io/`)
    - cvc5 is an efficient open-source automatic theorem prover for Satisfiability Modulo Theories (SMT) problems. It can be used to prove the satisfiability (or, dually, the validity) of first-order formulas with respect to (combinations of) a variety of useful background theories.

btor

- The btor engine works with btor2 files. It supports the following solvers:
  - btormc: `https://github.com/Boolector/boolector`
  - pono: `https://github.com/stanford-centaur/pono`

aiger

- The aiger engine (`https://github.com/arminbiere/aiger`) offers two solvers:
  - aigbmc
  - avy
  - rIC3
  - suprove (quite good for proofs)

abc

- The abc engine (`https://github.com/berkeley-abc/abc`) offers three solvers:
  - pdr (most used)
  - bmc3
  - sim3

Choosing an engine/solver

- Select for the task: BMC for bounded bug-finding, cover for a witness, and a prove-capable engine for an unbounded safety result
- Start with a supported, installed pair; for example: `abc bmc3` (BMC), `smtbmc z3` (BMC/cover), or `abc pdr` (prove)
- Use `code/engine/check_engines.py` to smoke-test installed combinations
- For this design, compare candidates under the same model, properties, assumptions, bound/timeout, and resources; record status, runtime, memory, and useful traces
- Choose based on measured results, not a claim that one solver is universally fastest or best. A smoke-test PASS does not guarantee convergence on a real design

Prove vs BMC

- Use BMC to search for counterexamples within a chosen horizon (for example, through a protocol latency or counter wrap)
  - A deeper bound can expose later bugs, but costs more resources; a bounded PASS says nothing beyond that bound
- Use a proof-capable engine when you need an unbounded safety result
  - A completed proof covers all reachable executions of the encoded model under its assumptions
  - A proof attempt may be harder or fail to converge; UNKNOWN or timeout is not a counterexample
- They are complementary: start with BMC for fast bounded bug-finding, then try proof where unbounded assurance is needed

Conclusion

Choose the check that answers the verification question: bounded search for bugs within a horizon; proof for an unbounded guarantee when it converges.

- Compare supported engines on the same model, properties, and assumptions; measure status, runtime, memory, and counterexample depth
  - Choose based on evidence for this design, not a universal “best” solver
  - SBY’s `-autotune` can help explore configurations; review the resulting tasks and tool availability